

Reading Model Results

A general framework for diagnosing what a model's numbers are actually telling you — not specific to any one project, dataset, or result.

A model's output numbers are a symptom, not a diagnosis. The same accuracy value can mean 'working well' or 'completely broken,' depending entirely on what's happening underneath it. This is a general way of reading results to tell those two situations apart.

1. Why a single number is never enough

Accuracy answers exactly one question: what fraction of predictions were correct? It says nothing about which kind of mistake a model makes, and it can be badly misleading when the two classes aren't equally common. A model that always predicts the majority class can score high accuracy while being completely useless — this is sometimes called the **accuracy paradox**. The fix isn't a better single number; it's looking at more than one number at once, and understanding what each is actually sensitive to.

2. The four core metrics, and what each one actually catches

Metric	Answers	Blind to
Accuracy	What fraction of all predictions were correct?	Class imbalance — can look great while ignoring the rare class entirely.
Precision	Of everything flagged positive, how much really was?	Says nothing about positives the model missed entirely.
Recall	Of all real positives, how many did the model catch?	Says nothing about false alarms — can be perfect by predicting positive for everyone.
F1-score	A single balance point between precision and recall.	Can hide which of the two is actually weak; always check its two parents.

3. Reading precision and recall together as a signature

The two numbers together form a pattern, and specific patterns point toward specific likely causes. This is where real diagnosis happens — not in either number alone.

Pattern	Likely meaning
---------	----------------

Recall = 1.00, precision $\approx$ base rate of the positive class	The model is predicting the positive class for almost everything. It has not learned to discriminate — it's guessing the same answer regardless of input.
Precision high, recall low	The model is cautious : when it commits to a positive prediction it's usually right, but it's missing many real positives. Often seen with a decision threshold set too conservatively, or with a class that's genuinely hard to detect.
Precision low, recall high	The model over-predicts positives : it catches most real cases but with many false alarms alongside them.
Both high	Genuine discrimination — the strongest evidence that real, usable signal is being captured, not an artifact.
Both hover near chance level, but not identical every time	The model may be weakly picking something up, or the features may simply carry very little real signal for this task.

4. The average hides the story — always look at the distribution

A single overall score (mean accuracy, mean F1) collapses many individual results into one number, and that collapse can hide exactly the pattern you need to see. Two very different situations can produce the identical average:

- **Consistently mediocre**: every individual result is close to the mean. This suggests a genuine, uniform limitation — the model (or its features) may simply not be strong enough, across the board.
- **Split personality**: some individual results are excellent, others are near-random, averaging out to something moderate. This suggests the model works under certain conditions but not others — which is often more fixable, because it points to a specific, identifiable difference between the 'working' and 'failing' cases worth investigating directly.

When evaluation naturally breaks into groups (different subjects, different sessions, different time periods), always inspect the per-group results individually before trusting the average of them.

5. Degenerate patterns worth recognizing on sight

- **Majority-class collapse**: the model predicts one class almost always, regardless of input. Recognizable from the precision/recall signature above. Common causes: severe class imbalance, not enough genuinely informative features, or a target the model can't actually predict from the given inputs.

- **Chance-level performance:** results hover around what pure random guessing would achieve (50% accuracy for a balanced binary problem). This suggests the features may carry little to no real signal for this specific task, not that the model implementation is broken.
- **Suspiciously perfect performance:** near-100% scores, especially early in a project, are usually a red flag rather than good news. Common causes: information from the answer leaking into the input features, or evaluating on data too similar to what was trained on (see the LOSO cross-validation reference note for a concrete version of this).
- **Wildly inconsistent results with no evaluation split changed:** can indicate an unstable model (too little training data for how flexible the model is) or a bug in how data is being shuffled or grouped between runs.

6. What genuinely trustworthy results tend to look like

- Both precision and recall are meaningfully above chance level — not just one of them.
- The model actually outputs both classes somewhere in its predictions, not just one, when both are genuinely present in the truth.
- Performance is reasonably consistent across different subsets of the data (different people, different time periods, different sessions) — wild swings deserve an explanation, not just a shrug.
- The result is not implausibly better than what the problem's inherent difficulty would suggest.
- The evaluation setup itself has been checked for leakage — the test data was never available to the model, directly or indirectly, during training.

7. A general diagnostic checklist

- Have I looked at precision and recall separately, not just accuracy or F1 alone?
- Have I looked at the actual predicted label distribution, not just the summary metrics?
- Have I inspected individual group/fold results, not just the overall average?
- Does the pattern I'm seeing match a known degenerate signature (majority-class collapse, chance level, suspiciously perfect)?
- If results vary a lot across groups, what's actually different about the groups that do well versus the ones that don't?
- Is there any way information about the answer could have reached the model indirectly, through the features or the way data was split?
- Would a domain expert find this result plausible, given how hard the underlying problem actually is?

The goal of reading results this way isn't to find a reason to trust a good-looking number — it's to find out what's actually happening underneath it, whether that turns out to be reassuring or not. A well-diagnosed failure is more valuable than an unexamined success.